



BANNARI AMMAN INSTITUTE OF TECHNOLOGY

An Autonomous Institution Affiliated to Anna University Chennai - Approved by AICTE - Accredited by NAAC with "A+" Grade

SATHYAMANGALAM - 638 401 ERODE DISTRICT TAMIL NADU INDIA

Ph: 04295-226000 / 221289 Fax: 04295-226666 E-mail: stayahead@bitsathy.ac.in Web: www.bitsathy.ac.in

22AI503/22AM503 MACHINE LEARNING

UNIT III – CLASSIFICATION

Support Vector Machines - Linear Learning Machines

1. Introduction

1.1 Definition

Support Vector Machines (SVM) are supervised, discriminative learning machines that find the optimal separating hyperplane between classes by maximising the margin — the distance between the hyperplane and the nearest data points of each class. This topic covers the linear formulation; kernel-based extensions to non-linear boundaries are covered in the next topic.

Definition:

Linear Learning Machine: a model that computes a decision function as a linear combination of input features: $f(x) = w \cdot x + b$.

Maximum Margin: SVM specifically seeks the hyperplane that maximises the distance to the nearest training points of either class — improving generalisation over arbitrary separating hyperplanes.

Support Vectors: the (typically few) training points closest to the decision boundary that alone determine the hyperplane's position.

1.2 Importance in Machine Learning

SVMs were the dominant classification technique throughout the 1990s-2000s, particularly for high-dimensional problems like text classification and bioinformatics, due to strong theoretical guarantees (Vapnik-Chervonenkis theory) and excellent generalisation even with limited training data. They remain a standard tool today, especially when training data is scarce relative to feature count.

1.3 Role in the Classification Pipeline

Linear SVM serves as a strong baseline for linearly (or near-linearly) separable problems and as the conceptual and mathematical foundation for kernel SVM (next topic), which extends the same margin-maximisation principle into non-linear feature spaces.

2. Learning Outcomes

Understand

- The concept of a maximum-margin hyperplane and why it generalises better than arbitrary separators
- The distinction between hard-margin and soft-margin SVM formulations
- The primal and dual optimisation formulations and their relationship

Apply

- Implement linear SVM classification using scikit-learn (LinearSVC, SVC with linear kernel)

- Tune the regularisation parameter C via cross-validation
- Identify support vectors from a trained model and interpret their role

Analyze

- Evaluate the bias-variance trade-off as a function of C
- Compare hinge loss to other loss functions (0-1 loss, logistic loss)
- Diagnose when a problem requires soft-margin formulation versus hard-margin

3. Detailed Technical Concepts

3.1 Linear Learning Machines

A linear learning machine computes predictions via a linear decision function: $f(x) = w \cdot x + b$, where w is the weight vector (perpendicular to the decision hyperplane) and b is the bias (offset). Classification is performed by the sign of $f(x)$: predict class +1 if $f(x) \geq 0$, class -1 otherwise. Many classifiers (Logistic Regression, Perceptron, Linear SVM) share this linear functional form but differ in how they choose w and b (i.e., which loss function and optimisation objective they use).

3.2 The Maximum Margin Principle

Among all hyperplanes that correctly separate the training data, infinitely many exist (assuming the data is linearly separable). SVM specifically selects the one hyperplane that maximises the margin — the minimum distance from the hyperplane to the nearest training point of either class. This choice is motivated by statistical learning theory: wider margins correspond to better generalisation bounds on unseen data.

The margin width is given by $2/\|w\|$. Maximising the margin is therefore equivalent to minimising $\|w\|^2$ (a convenient convex quadratic objective).

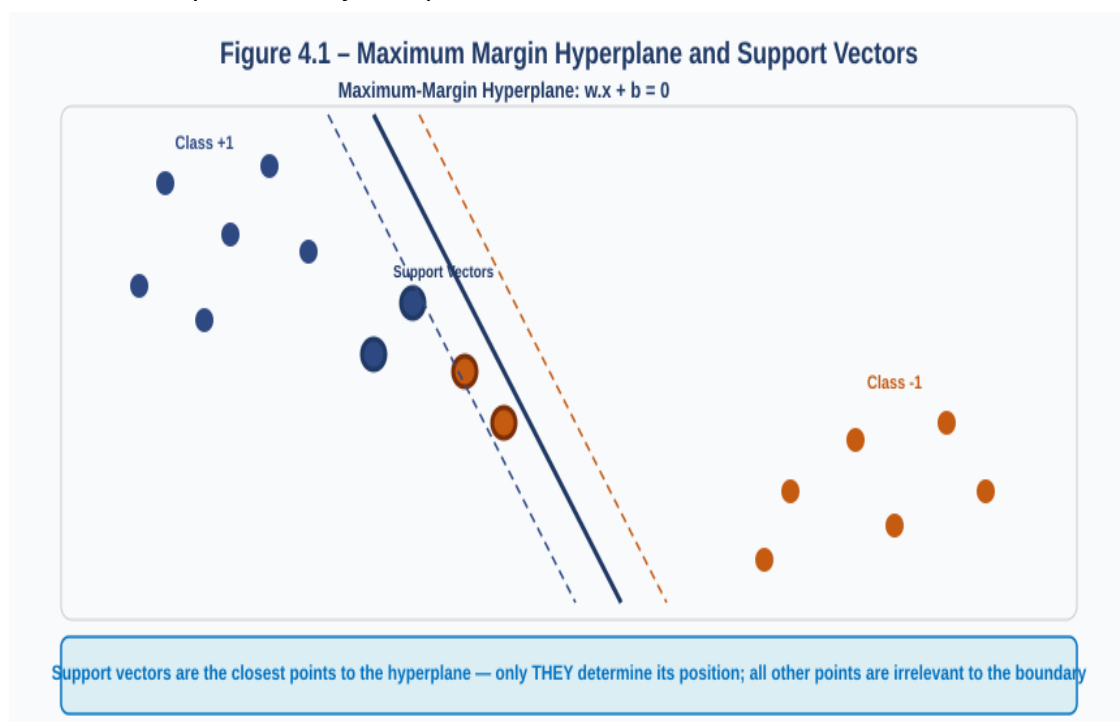


Figure 4.1 – Maximum Margin Hyperplane and Support Vectors

3.3 Support Vectors

Support vectors are the training points that lie exactly on the margin boundary (or, in the soft-margin case, that violate the margin). These points alone determine the position and orientation of the hyperplane — all other training points could be removed without changing the solution. This property gives SVM both its name and a useful interpretability/efficiency benefit: predictions only depend on a (typically small) subset of "informative" training examples.

3.4 Hard Margin SVM

For linearly separable data, the hard-margin SVM solves:

minimise $(1/2)||w||^2$ subject to: $y_i(w \cdot x_i + b) \geq 1$ for all i

Here y_i in $\{-1, +1\}$ is the class label, and the constraint ensures every training point is correctly classified and lies at least at margin-distance from the hyperplane. This formulation requires the data to be perfectly linearly separable — any single mislabelled point, outlier, or overlapping class region makes the problem infeasible (no solution exists).

3.5 Soft Margin SVM

Real-world data is rarely perfectly separable. The soft-margin SVM introduces non-negative slack variables ξ_i for each training point, allowing controlled margin violations:

minimise $(1/2)||w||^2 + C \times \sum(\xi_i)$ subject to: $y_i(w \cdot x_i + b) \geq 1 - \xi_i$, $\xi_i \geq 0$

The regularisation parameter C controls the trade-off between maximising the margin and minimising classification errors on the training set:

- Large C : heavily penalises margin violations — narrower margin, fewer training errors tolerated, risk of overfitting (especially with noisy data).
- Small C : tolerates more margin violations — wider margin, more robust to noise/outliers, risk of underfitting if too small.

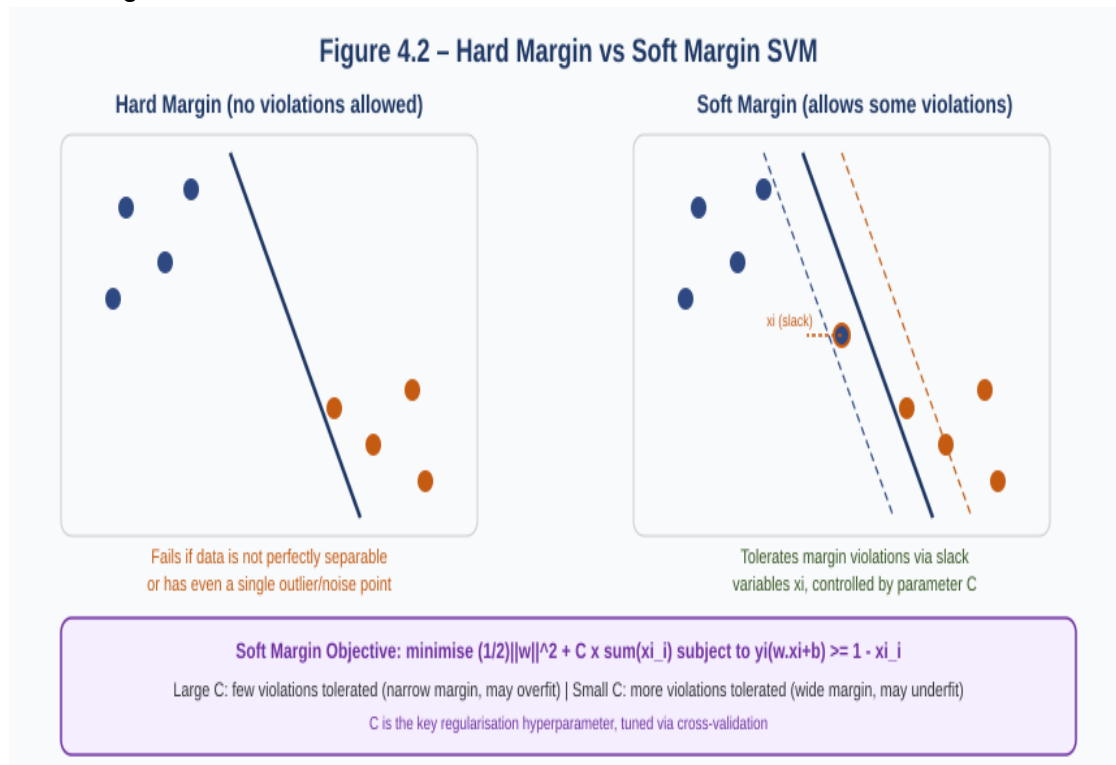


Figure 4.2 – Hard Margin vs Soft Margin SVM with the C Regularisation Parameter

3.6 Primal and Dual Formulations

The SVM optimisation problem can be solved in two equivalent forms:

3.6.1 Primal Form

Directly optimises the weight vector w and bias b in the original d -dimensional feature space. Efficient when the number of features d is small relative to the number of training examples n .

3.6.2 Dual Form

Using Lagrange multipliers a_i (one per training point), the problem is reformulated as:

maximise $\sum(a_i) - (1/2) \sum_i \sum_j a_i a_j y_i y_j (x_i \cdot x_j)$ subject to: $a_i \geq 0$, $\sum(a_i y_i) = 0$

Critically, the dual formulation depends on the training data ONLY through pairwise dot products $(x_i \cdot x_j)$ — never on the individual feature vectors directly. This property is what enables the kernel trick (covered in the next topic): replacing the dot product with a more general kernel function allows SVM to operate implicitly in much higher-dimensional (even infinite-dimensional) feature spaces without ever explicitly computing the transformed coordinates.

Points with $a_i > 0$ in the dual solution are exactly the support vectors; points with $a_i = 0$ do not influence the final decision boundary at all.

3.7 Hinge Loss

The soft-margin SVM objective can be equivalently expressed using the hinge loss function:

$$L(y, f(x)) = \max(0, 1 - y \cdot f(x))$$

This loss is zero when a point is correctly classified and lies beyond the margin ($y \cdot f(x) \geq 1$), and grows linearly as the point moves inside the margin or is misclassified. The full SVM objective becomes: minimise $\sum_i L(y_i, f(x_i)) + \lambda \|w\|^2$ — a regularised empirical risk minimisation, placing SVM within the broader family of regularised linear models alongside Ridge and LASSO regression.

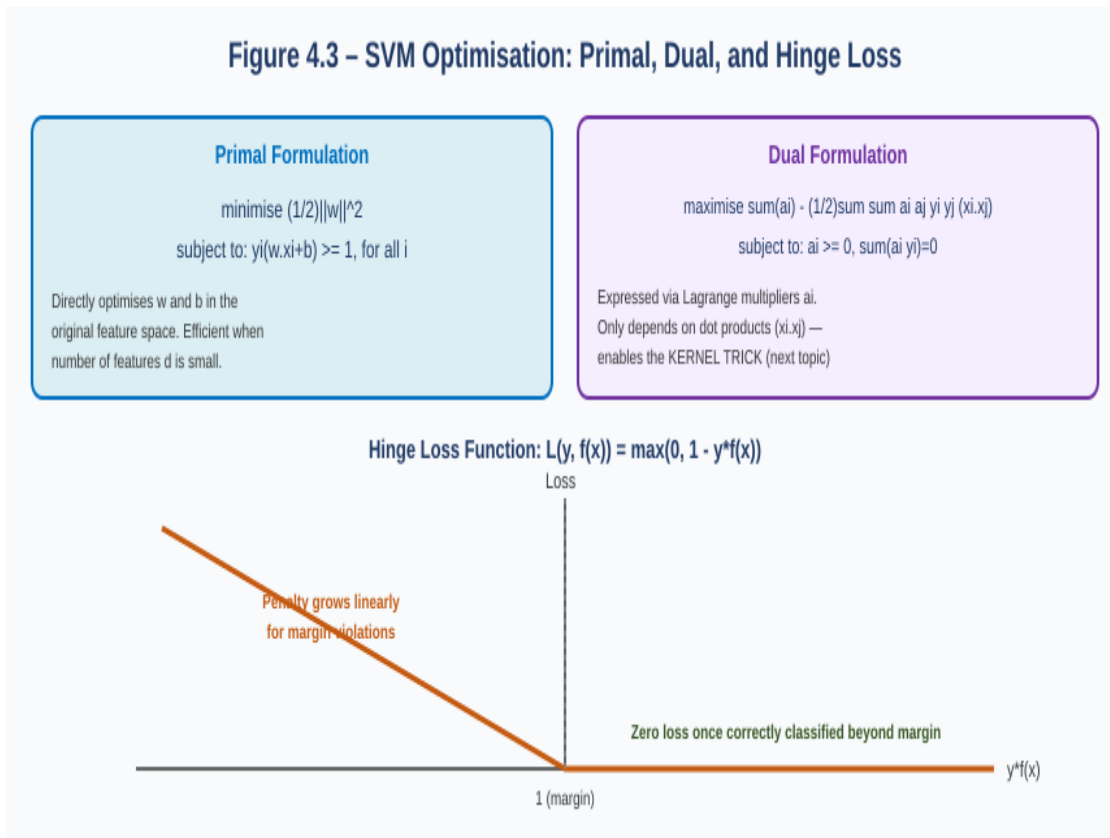


Figure 4.3 – SVM Optimisation: Primal Formulation, Dual Formulation, and Hinge Loss

3.8 SVM vs Other Linear Classifiers

Classifier	Loss Function	Key Property
Linear SVM	Hinge loss	Maximises margin; sparse solution (few support vectors)
Logistic Regression	Log loss (cross-entropy)	Probabilistic outputs; all points contribute to the boundary
Perceptron	0-1 / Perceptron loss	Finds ANY separating hyperplane, no margin maximisation
LDA	Gaussian likelihood-based	Assumes Gaussian class-conditional distributions

4. Architecture / SVM Training Workflow

Step	Action	Method / Code	Output
1	Scale features	StandardScaler().fit_transform(X)	Standardised feature matrix
2	Choose margin type	Hard (rare) vs Soft margin (default)	Formulation choice
3	Set up dual optimisation	Quadratic programming solver (SMO)	Lagrange multipliers a_i
4	Tune C via cross-validation	GridSearchCV over $C=[0.01...100]$	Optimal regularisation strength
5	Identify support vectors	model.support_vectors_	Subset of training points defining boundary
6	Predict via decision function	$\text{sign}(w \cdot x + b)$	Predicted class label
7	Validate on held-out test set	accuracy_score, classification_report	Final performance metrics

5. Industry Tools and Technologies

5.1 Commercial Tools

Tool	Vendor	SVM Features
SPSS Modeler	IBM	SVM node with auto-C tuning
SAS Enterprise Miner	SAS	PROC HPSVM for large-scale linear SVM
MATLAB Statistics Toolbox	MathWorks	fitsvm with linear and kernel options
RapidMiner	RapidMiner	SVM operator (LibSVM-based) with kernel selection
Azure ML Studio	Microsoft	Two-Class SVM module

5.2 Open-Source Tools

- scikit-learn: LinearSVC (optimised for linear kernel, liblinear backend), SVC(kernel="linear")
- LIBSVM / LIBLINEAR: foundational C++ libraries underlying many SVM implementations across languages
- e1071 (R): svm() function with linear and other kernel options
- Spark MLlib: LinearSVC for distributed large-scale linear SVM training
- Vowpal Wabbit: highly efficient online learning supporting hinge-loss linear models at massive scale

6. Real-World Applications

Domain	Application	Why Linear SVM	Outcome
Text Classification	Document/email categorisation	High-dim sparse text data is often linearly separable	Fast, accurate even with huge vocabularies
Bioinformatics	Gene expression cancer classification	Few samples, many features ($d \gg n$) regime	Strong generalisation despite limited data
Finance	Credit approval / default prediction	Interpretable linear decision boundary	Regulatory-compliant transparent decisions
Image Classification (pre-deep-learning)	Handwritten digit/face recognition on extracted features	Robust margin-based generalisation	Was state-of-the-art before CNNs
Customer Churn	Predicting subscription cancellation	Few but well-engineered features	Strong baseline before trying ensembles

7. Advantages and Limitations

Advantages	Limitations
Strong theoretical guarantees (margin maximisation, VC theory)	Linear SVM cannot capture non-linear decision boundaries (needs kernels)
Effective in high-dimensional spaces, even $d \gg n$	Choice of C requires careful tuning; sensitive to hyperparameter
Memory-efficient at prediction time (only support vectors matter)	Training time scales poorly for very large n with non-linear kernels (less so for linear)
Robust to overfitting via margin maximisation and regularisation	No native probabilistic output (requires Platt scaling for calibrated probabilities)
Sparse solution — few support vectors define the entire model	Sensitive to feature scaling; sensitive to class imbalance without weighting

8. Case Study

CASE STUDY

Case: Reuters-21578 Text Categorisation — Linear SVM Benchmark (Joachims, 1998)

Problem: Thorsten Joachims' seminal 1998 study established SVM as the leading method for text classification, benchmarked on the Reuters-21578 news categorisation dataset.

Dataset: 21,578 Reuters news articles, ~10,000 TF-IDF word features per document, 90 topic categories (e.g., "earnings", "acquisitions", "crude oil").

Approach: Applied Linear SVM (no kernel needed) directly on the high-dimensional sparse TF-IDF feature vectors, leveraging the fact that text data is typically near-linearly separable once vectorised.

Comparison: Benchmarked against Naive Bayes, k-NN, Decision Trees (C4.5), and Rocchio classifiers across all 90 categories.

Findings: Linear SVM achieved the highest micro-averaged F1 score (0.864) across all 90 categories, outperforming Naive Bayes (0.795), k-NN (0.823), and C4.5 decision trees (0.794). SVM was particularly strong on categories with few positive training examples — a direct benefit of the margin-maximisation principle generalising well from limited data.

Conclusion: This study, alongside similar results across many domains, established Linear SVM as the dominant high-dimensional text classification method for over a decade, prized for combining strong accuracy with computational efficiency on sparse, high-dimensional bag-of-words/TF-IDF data.

9. Scenario-Based Learning

SCENARIO 1

SCENARIO 1: Choosing C for a Noisy Medical Dataset

Background: A hospital builds a linear SVM to classify tumour malignancy (benign/malignant) using 30 cytology features from the Wisconsin Breast Cancer dataset. Some data entry errors are suspected in the historical records.

Technical Environment: Python, scikit-learn LinearSVC, StandardScaler, 569 patient samples.

Problem Statement: With default $C=1.0$, the model achieves only 89% test accuracy with high variance across cross-validation folds (range 84%-94%), suggesting instability.

Available Evidence: Visual inspection of a few misclassified training points reveals 3-4 likely data-entry errors (e.g., a benign tumour with cytology values typical of malignant cases) acting as label noise.

Investigation: Grid search C over [0.001, 0.01, 0.1, 1, 10, 100] with 10-fold cross-validation, plotting mean accuracy and variance for each C .

Analysis: $C=0.1$ (smaller than default) gives a wider margin, tolerating the few noisy/mislabelled points as acceptable margin violations rather than forcing the boundary to contort around them.

Findings: With $C=0.1$: test accuracy improved to 96.8% with much lower fold-to-fold variance (93%-98% range). The wider margin made the model robust to the suspected label noise.

Recommended Solution: When data quality is uncertain or training data is noisy/small, prefer smaller C values (more regularisation, wider margin) and always validate stability via cross-validation variance, not just mean accuracy.

SCENARIO 2

SCENARIO 2: Unscaled Features Distort the Margin

Background: A fintech company builds a linear SVM credit-default classifier using features: annual_income (range \$10K-\$500K), num_late_payments (range 0-20), credit_utilization_ratio (range 0-1).

Technical Environment: Python, scikit-learn LinearSVC (no scaling applied initially), 100,000 loan records.

Problem Statement: The trained model essentially ignores credit_utilization_ratio and num_late_payments, with the decision boundary almost entirely determined by annual_income — counter to known credit-risk domain knowledge where late payments are highly predictive.

Available Evidence: Examining the learned weight vector w , the coefficient magnitude for annual_income

is ~50,000x larger than for `credit_utilization_ratio`, purely reflecting the difference in input feature scale, not true relative importance.

Investigation: SVM's margin-maximisation objective operates on raw Euclidean geometry — unscaled features with large numeric ranges dominate $\|w\|^2$ minimisation and the margin computation.

Analysis: Apply `StandardScaler` to all numeric features before training, ensuring each contributes comparably to the margin geometry.

Findings: After scaling, the learned weight vector showed appropriately balanced contributions:

`num_late_payments` and `credit_utilization_ratio` became the top two predictors by coefficient magnitude, matching domain expectations. Model AUC improved from 0.71 to 0.85.

Recommended Solution: Always apply feature scaling (`StandardScaler`) before training any SVM — like kNN, SVM is fundamentally a geometric/distance-based method and is highly sensitive to feature scale.

SCENARIO 3

SCENARIO 3: Hard Margin SVM Fails on Overlapping Classes

Background: A quality-control team at a manufacturing plant attempts to use a hard-margin SVM to classify products as Pass/Fail based on two sensor measurements (vibration, temperature).

Technical Environment: Python, manual hard-margin SVM implementation (constraint: $y_i(w \cdot x_i + b) \geq 1$ for all i , no slack).

Problem Statement: The optimisation solver fails to converge / reports infeasibility for the full dataset of 5,000 products, even though the vast majority of points are clearly separable by eye.

Available Evidence: Scatter plot reveals 12 borderline products (out of 5,000) where Pass and Fail classes slightly overlap in the measurement space — likely due to measurement noise or genuinely ambiguous near-threshold products.

Investigation: Hard-margin SVM requires perfect linear separability; even a single non-separable point makes the constraint set infeasible (no w , b satisfies all constraints simultaneously).

Analysis: Switch to soft-margin SVM with slack variables, allowing the 12 overlapping points to violate the margin (at a tunable cost via C) while still finding a sensible boundary for the other 4,988 points.

Findings: Soft-margin SVM with $C=1.0$ converged successfully: classified 4,985 of 5,000 products correctly (99.7% training accuracy), with the 12 ambiguous points correctly identified as the only margin violations — exactly matching the visual overlap region.

Recommended Solution: In nearly all real-world applications, prefer soft-margin SVM over hard-margin. Hard-margin SVM is primarily a theoretical/pedagogical baseline; it is rarely usable on real data due to its brittleness to any class overlap or label noise.

10. Common Errors and Troubleshooting

Error	Cause	Impact	Fix
Using hard-margin SVM on real data	Assuming perfect linear separability	Optimisation infeasible / fails to converge	Use soft-margin SVM (default in scikit-learn) with tunable C
Skipping feature scaling	Treating SVM like tree-based models	Margin dominated by high-range features	Always <code>StandardScale</code> features before SVM
Default C without tuning	Not running cross-validation on C	Suboptimal bias-variance trade-off	Grid/random search C over a log-scale range
Interpreting raw decision scores as probabilities	Confusing distance-to-hyperplane with confidence	Miscalibrated probability estimates	Use Platt scaling (probability=True in sklearn) for calibrated probabilities
Applying linear SVM to clearly non-linear data	Not checking decision boundary fit visually	Underfitting, poor accuracy	Switch to kernel SVM (see next topic) when classes are not linearly separable

11. Key Terminologies

Term	Definition
Hyperplane	A decision boundary of the form $w \cdot x + b = 0$ separating classes in feature space
Margin	The distance between the hyperplane and the nearest training points of either class
Support Vectors	Training points closest to (or violating) the margin; alone determine the hyperplane
Hard Margin	SVM formulation requiring perfect linear separability with no margin violations
Soft Margin	SVM formulation allowing controlled margin violations via slack variables ξ_i
Slack Variable (ξ_i)	Non-negative variable measuring how much a point violates the margin constraint
C (Regularisation Parameter)	Controls trade-off between margin width and tolerance for misclassification
Primal Problem	SVM optimisation expressed directly in terms of weight vector w and bias b
Dual Problem	SVM optimisation expressed via Lagrange multipliers, depending only on dot products
Lagrange Multiplier (α_i)	Dual variable per training point; positive only for support vectors
Hinge Loss	Loss function $L = \max(0, 1 - y \cdot f(x))$ used in SVM's regularised risk formulation
Linear Separability	Property of data where classes can be perfectly divided by a straight hyperplane

12. Topic Summary

UNIT III – Topic 4

SUPPORT VECTOR MACHINES — LINEAR LEARNING MACHINES — SUMMARY

- SVM finds the hyperplane $w \cdot x + b = 0$ that maximises the margin (distance to nearest points), motivated by stronger generalisation bounds.
- Support vectors are the (few) points closest to the boundary that alone determine its position; all other points are irrelevant to the solution.
- Hard margin requires perfect separability (rarely usable in practice); soft margin (default) allows controlled violations via slack variables ξ_i and regularisation parameter C .
- C trade-off: large C = narrow margin, fewer training errors tolerated (overfitting risk); small C = wide margin, more tolerant (underfitting risk if too small).
- Primal formulation optimises w, b directly; dual formulation (via Lagrange multipliers) depends only on pairwise dot products — the foundation for the kernel trick (next topic).
- The soft-margin objective is equivalent to minimising hinge loss plus an L2 regularisation term: a regularised empirical risk minimisation problem.
- SVM is highly sensitive to feature scaling — always StandardScale before training.
- Tools: scikit-learn (LinearSVC, SVC(kernel="linear")), LIBSVM/LIBLINEAR, Spark MLlib, Vowpal Wabbit.